

Atlassian App Analytics

Complete Project Walkthrough and Technical Documentation

Dr. Joseph Njeri
Elsa-Brandström-Str. 15a
76228 Karlsruhe, Germany
nkarianjahi@gmail.com

May 11, 2026

Contents

1 Overview	4
2 The Business Problem	4
3 Project Motivation	4
4 Repository and Local Setup	4
5 PostgreSQL Setup	5
6 Django Project Creation	5
7 Django Settings	5
8 Data Model	6
8.1 Customer	6
8.2 UsageEvent	6
8.3 CustomerHealth	7
8.4 CustomerHealthSnapshot	7
8.5 CSVUploadLog	7
9 Migrations	7

10 Customer Health Scoring	7
11 Machine Learning Pipeline	8
12 Feature Importance	8
13 Historical Snapshots	9
14 Insights, Alerts, and Recommendations	9
15 Forecasting	9
16 CSV Upload Workflow	10
17 Upload Audit Logging	11
18 Dashboard	11
19 Customer Detail Page	12
20 REST API Layer	12
21 Management Commands	13
22 Deployment Files	13
22.1 Procfile	13
22.2 build.sh	13
22.3 .env.example	13
23 Deployment on Render	13
24 Technology Stack	14
25 Business Questions Answered	14
26 Skills Demonstrated	14
27 Why This Is More Than CRUD	15
28 Future Enhancements	15

29 Conclusion

15

1 Overview

Atlassian App Analytics is a full-stack customer intelligence platform that ingests customer and usage-event data from CSV files, stores it in PostgreSQL, calculates customer health scores, predicts churn probability using machine learning, tracks historical trends, generates alerts and recommendations, exposes REST API endpoints, and visualizes insights in an interactive dashboard.

2 The Business Problem

Software vendors in the Atlassian Marketplace often collect large volumes of product usage data but still struggle to answer practical questions:

- Which customers are healthy?
- Which customers are at risk of churn?
- Why is risk increasing?
- What should customer success teams do?
- What may happen next?

The goal of this project is to convert raw usage events into actionable customer intelligence.

3 Project Motivation

The project was inspired by Atlassian Marketplace vendors such as codefortynine GmbH in Karlsruhe, Germany. It is not affiliated with codefortynine, but it is designed around the kind of customer analytics that could support Atlassian cloud apps such as Deep Clone for Jira, External Data for Jira Fields, Quick Filters for Jira Dashboards, Merge Agent for Jira, and Snipe-IT for Jira.

4 Repository and Local Setup

```
git clone https://github.com/karianjahi/atlassian-app-analytics.git
cd atlassian-app-analytics

python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip

pip install django pandas scikit-learn joblib \
    django-rest-framework psycopg2-binary \
    gunicorn whitenoise dj-database-url python-dotenv
```

```
pip freeze > requirements.txt
```

5 PostgreSQL Setup

```
sudo apt update
sudo apt install postgresql postgresql-contrib
sudo systemctl start postgresql
sudo -u postgres psql
```

```
CREATE DATABASE atlassian;
CREATE USER analytics_user WITH PASSWORD 'your_secure_password';

ALTER ROLE analytics_user SET client_encoding TO 'utf8';
ALTER ROLE analytics_user SET default_transaction_isolation TO 'read committed';
ALTER ROLE analytics_user SET timezone TO 'UTC';

GRANT ALL PRIVILEGES ON DATABASE atlassian TO analytics_user;
ALTER DATABASE atlassian OWNER TO analytics_user;
```

6 Django Project Creation

```
django-admin startproject core
cd core
python manage.py startapp analytics
```

```
atlassian-app-analytics/
|-- core/
| |-- manage.py
| |-- core/
| |-- analytics/
|-- requirements.txt
|-- Procfile
|-- build.sh
|-- .env.example
|-- README.md
```

7 Django Settings

```
INSTALLED_APPS = [
    ...,
    'rest_framework',
```

```
'analytics',  
]
```

```
ALLOWED_HOSTS = os.environ.get(  
    'ALLOWED_HOSTS',  
    '127.0.0.1,localhost'  
) .split(',')
```

```
DATABASES = {  
    'default': dj_database_url.config(  
        default='sqlite:///db.sqlite3'  
    )  
}
```

```
STATIC_URL = '/static/'  
STATIC_ROOT = BASE_DIR / 'staticfiles'
```

8 Data Model

8.1 Customer

The `Customer` model represents a company using an Atlassian Marketplace app.

- `company_name`
- `country`
- `company_size`
- `license_tier`
- `app_name`
- `installed_at`
- `created_at`

8.2 UsageEvent

The `UsageEvent` model stores product activity.

- `customer`
- `event_type`
- `timestamp`
- `metadata`
- `created_at`

8.3 CustomerHealth

The CustomerHealth model stores latest analytics results.

- usage_score
- feature_adoption_score
- reliability_score
- support_score
- health_score
- churn_risk
- risk_label
- did_churn
- ml_churn_probability
- calculated_at

8.4 CustomerHealthSnapshot

The CustomerHealthSnapshot model stores historical health values over time.

8.5 CSVUploadLog

The CSVUploadLog model records each CSV upload for auditability.

9 Migrations

```
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
```

10 Customer Health Scoring

The health score combines usage, adoption, reliability, and support.

$$\text{Health Score} = 0.40(\text{Usage}) + 0.25(\text{Feature Adoption}) + 0.20(\text{Reliability}) + 0.15(\text{Support})$$

Score Range	Label
80–100	Healthy
50–79	Watch
0–49	High Risk

Rule-based churn risk is calculated as:

$$\text{Churn Risk} = 100 - \text{Health Score}$$

11 Machine Learning Pipeline

The machine learning layer uses scikit-learn Logistic Regression to predict churn probability.

1. Extract training data from CustomerHealth.
2. Build a pandas DataFrame.
3. Use health metrics and company size as features.
4. Create a synthetic target label.
5. Split data into training and test sets.
6. Train a Logistic Regression model.
7. Evaluate accuracy.
8. Save the model using joblib.
9. Predict churn probabilities.

Synthetic churn label:

```
did_churn = health_score < 40
```

Saved model:

```
analytics/churn_model.joblib
```

12 Feature Importance

The system exposes model coefficients to explain which features influence churn prediction.

- Positive coefficient: increases churn probability.
- Negative coefficient: decreases churn probability.
- Larger absolute value: stronger influence.

13 Historical Snapshots

Every analytics refresh creates a `CustomerHealthSnapshot`. Each snapshot stores:

- component scores
- health score
- churn risk
- ML churn probability
- risk label
- timestamp

This enables historical trend analysis and auditability.

14 Insights, Alerts, and Recommendations

The platform generates plain-English insights and recommended actions.

Examples include:

- Low recent product usage.
- Limited feature adoption.
- High number of failed events.
- High support activity.
- Schedule a customer check-in.
- Investigate recent technical errors.

15 Forecasting

The function `forecast_next_health_score(customer)` estimates the next customer health score from recent historical snapshots.

Example sequence:

$80 \rightarrow 75 \rightarrow 70 \rightarrow 65 \rightarrow 60$

Average change:

$$\frac{60 - 80}{4} = -5$$

Forecast:

$$60 + (-5) = 55$$

The forecasted next health score is therefore 55.

16 CSV Upload Workflow

The upload page accepts CSV files with the following required columns:

- company_name
- app_name
- country
- company_size
- license_tier
- installed_at
- event_type
- timestamp
- metadata

Workflow:

1. Validate file structure.
2. Read the CSV with pandas.
3. Check required columns.
4. Create or reuse customers.
5. Insert usage events.
6. Skip duplicate events.
7. Recalculate customer health.
8. Update ML predictions.

9. Create historical snapshots.
10. Record upload log.
11. Display success summary.

17 Upload Audit Logging

Each upload records:

- filename
- upload timestamp
- customers created
- events imported
- duplicates skipped
- invalid rows skipped

18 Dashboard

The dashboard displays:

- total customers
- average health score
- healthy, watch, and high-risk counts
- ML model accuracy
- health distribution chart
- feature importance table
- top ML churn-risk customers
- top rule-based risk customers
- paginated customer table

19 Customer Detail Page

Each customer page shows:

- company profile
- health score breakdown
- churn probability
- recent events
- event distribution chart
- events-over-time chart
- health history chart
- alerts
- insights
- recommendations
- forecast

20 REST API Layer

Key endpoints include:

- `/api/customers/`
- `/api/dashboard/`
- `/api/customer-health/`
- `/api/ml/metrics/`
- `/api/ml/feature-importance/`
- `/api/customers/<id>/detail/`
- `/api/customers/<id>/health-history/`

The API supports pagination, filtering, and ordering.

21 Management Commands

```
python manage.py update_customer_health
python manage.py update_ml_churn
./update_analytics.sh
```

22 Deployment Files

22.1 Procfile

```
web: gunicorn core.wsgi:application
```

22.2 build.sh

```
#!/usr/bin/env bash
set -o errexit

pip install -r requirements.txt
python manage.py collectstatic --no-input
python manage.py migrate
```

22.3 .env.example

```
SECRET_KEY=change-me
DEBUG=False
ALLOWED_HOSTS=127.0.0.1,localhost
DATABASE_URL=postgresql://user:password@localhost:5432/atlassian
```

23 Deployment on Render

Render settings:

- Root Directory: core
- Build Command: ./build.sh
- Start Command: gunicorn core.wsgi:application

Live application:

<https://customer-health-app.onrender.com>

24 Technology Stack

Layer	Technology
Backend	Django, Python
API	Django REST Framework
Database	PostgreSQL
Data Processing	pandas
Machine Learning	scikit-learn, joblib
Frontend	HTML, CSS, JavaScript
Visualization	Chart.js
Deployment	Render, Unicorn, WhiteNoise

25 Business Questions Answered

The platform answers:

- What is customer health now?
- Which customers are likely to churn?
- Why is risk increasing?
- What actions should be taken?
- Is intervention urgent?
- What may happen next?

26 Skills Demonstrated

- Data engineering
- Backend development
- REST API design
- Machine learning
- Product analytics
- Frontend visualization
- Cloud deployment
- Auditability and reproducibility

27 Why This Is More Than CRUD

CRUD stands for Create, Read, Update, and Delete.

This project goes beyond CRUD by integrating:

- predictive analytics
- machine learning
- explainability
- forecasting
- alerting
- decision support
- deployment

28 Future Enhancements

- Authentication and role-based access
- Scheduled retraining
- Email notifications
- Real Atlassian API integration
- Docker and CI/CD
- Cohort analysis
- Forecast confidence intervals

29 Conclusion

Atlassian App Analytics evolved from a customer retention idea into a production-style customer intelligence platform.

It demonstrates how business understanding, software engineering, data science, and product thinking can be combined to create a practical and impactful solution.

Useful Links

- GitHub: <https://github.com/karianjahi/atlassian-app-analytics>
- Live Demo: <https://customer-health-app.onrender.com>
- Contact: nkarianjahi@gmail.com